External Entity Definition Sources

Previous: Entity Definitions | Documentation Home | Next: Module System

This guide explains how to replace the default `classpath:coredeux-entities.yml` loading behavior with an external source such as:

- a database
- a secrets manager
- a vault
- a remote configuration system

This document intentionally does not prescribe how the external system stores the data. The only assumption is:

- your code can obtain the complete YAML content as a `String`

When You Need This

The default `coredeux-core` setup loads entity definitions from a classpath resource:

- `coredeux.entities.config-location=classpath:coredeux-entities.yml`

That is the right default for:

- local development
- simple Spring Boot applications
- version-controlled entity definitions

You should consider an external source when you need:

- central configuration management
- per-environment dynamic definitions
- runtime-controlled configuration rollout
- security-controlled access to framework metadata
- operational separation between deployment artifact and entity config

Current Default Loading Path

The current loading pipeline in `coredeux-core` is:

1. Spring creates `EntityDefinitionRegistry`
2. the default config class reads a `Resource`
3. `YamlEntityDefinitionLoader` parses the YAML
4. `InMemoryEntityDefinitionRegistry` stores the parsed definitions

Relevant classes:

- `CoredeuxEntityDefinitionConfiguration`
- `YamlEntityDefinitionLoader`
- `InMemoryEntityDefinitionRegistry`

Important design point:

- the registry does not care where the YAML came from
- it only needs parsed `CoredeuxEntityDefinition` instances

That makes external-source integration straightforward.

Recommended Integration Pattern

The cleanest way to use an external source is:

1. read the YAML text from your external system
2. convert the text into an `InputStream`
3. parse it with the existing `EntityDefinitionLoader`
4. create an `InMemoryEntityDefinitionRegistry`
5. expose that registry as the Spring bean used by Coredeux

This lets you reuse the current YAML contract and validation rules without forking the loader.

What You Override

You do not need to change:

- `YamlEntityDefinitionLoader`
- `CoredeuxEntityDefinition`
- `CoredeuxStrategy`
- `CoredeuxService`

You only need to replace the default `EntityDefinitionRegistry` bean creation.

In practical terms:

- do not use the default resource-based registry bean
- provide your own `@Bean` of type `EntityDefinitionRegistry`

Example: External YAML Text Provider

First define a small abstraction in your application module.

```
public interface ExternalCoredeuxYamlProvider {

    String loadYamlText();
}
```

This provider can internally read from:

- a database
- AWS Secrets Manager
- HashiCorp Vault
- Azure Key Vault
- a remote config service

Coredeux does not need to know the source.

Example: Custom Registry Configuration

```
@Configuration
public class ExternalCoredeuxEntityDefinitionConfiguration {

    @Bean
    @Primary
    public EntityDefinitionRegistry entityDefinitionRegistry(
            ExternalCoredeuxYamlProvider yamlProvider,
            EntityDefinitionLoader loader) {
        String yamlText = yamlProvider.loadYamlText();
        InputStream inputStream = new ByteArrayInputStream(
                yamlText.getBytes(StandardCharsets.UTF_8));
        return new
InMemoryEntityDefinitionRegistry(loader.load(inputStream).getEntities());
    }
}
```

Why this works:

- `EntityDefinitionLoader` already knows how to parse Coredeux YAML
- `InMemoryEntityDefinitionRegistry` already knows how to hold the parsed model
- the rest of Coredeux only depends on `EntityDefinitionRegistry`

Example: Database-Backed Provider

The following example keeps the guide source-agnostic while showing the general pattern for database access.

```
@Component
public class DatabaseCoredeuxYamlProvider implements
ExternalCoredeuxYamlProvider {

    private final JdbcTemplate jdbcTemplate;

    public DatabaseCoredeuxYamlProvider(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    @Override
    public String loadYamlText() {
        String yaml = jdbcTemplate.queryForObject(
                "select yaml_text from coredeux_config where config_key = ?",
```

```
                String.class,
                "coredeux-entities");

        if (yaml == null || yaml.isBlank()) {
            throw new IllegalStateException("No Coredeux entity definition YAML
was returned from the database");
        }

        return yaml;
    }
}
```

The exact query and schema are application concerns, not Coredeux concerns.

Example: Secrets Manager Or Vault Provider

The same pattern works if your application already has a client that can read a secret value.

```
@Component
public class SecretStoreCoredeuxYamlProvider implements
ExternalCoredeuxYamlProvider {

    private final SecretClient secretClient;

    public SecretStoreCoredeuxYamlProvider(SecretClient secretClient) {
        this.secretClient = secretClient;
    }

    @Override
    public String loadYamlText() {
        String yaml = secretClient.readSecret("coredeux/entities-yaml");
        if (yaml == null || yaml.isBlank()) {
            throw new IllegalStateException("Secret store returned empty
Coredeux YAML");
        }
        return yaml;
    }
}
```

Again, the external client is application-specific.

Minimal End-to-End Example

The smallest possible override looks like this:

```
@Configuration
public class ExternalCoredeuxConfig {

    @Bean
    @Primary
    public EntityDefinitionRegistry entityDefinitionRegistry(
            EntityDefinitionLoader loader) {
        String yamlText = """
                coredeux:
                  entities:
                    - full-class-name: com.example.customer.Customer
                      identifier: pk
                      storage:
                        data-access-service: defaultCoredeuxJpaDataAccessService
                """;
```

```
        return new InMemoryEntityDefinitionRegistry(
                loader.load(new
ByteArrayInputStream(yamlText.getBytes(StandardCharsets.UTF_8))).getEntities());
    }
}
```

That example proves the only thing Coredeux really requires is valid YAML text.

Bean Precedence And Spring Wiring

The default `coredeux-core` config also declares an `EntityDefinitionRegistry` bean.

There are three common ways to make your custom bean win:

1. mark your custom bean with `@Primary`
2. disable the default config class from component scanning
3. replace the bean name explicitly in your application context

Recommended approach:

- use `@Primary`

That is usually the least invasive option in Spring Boot applications.

Validation And Error Behavior

Your custom external-source setup still uses the standard `YamlEntityDefinitionLoader`, so you keep current validation rules.

Examples of failures that will still be caught:

- empty YAML
- missing `coredeux.entities`
- missing `full-class-name`
- missing `identifier`
- missing `storage.data-access-service`
- malformed module config

That is a major advantage of reusing the built-in loader instead of parsing the YAML yourself.

Startup Timing

With this override, entity definitions are still loaded at application startup.

That means:

- the application fails fast if the YAML is invalid
- the application fails fast if the external source is unavailable
- the registry is stable for the lifetime of the application unless you build a refresh mechanism

yourself

This is usually the right default.

Runtime Refresh Considerations

The current `coredeux-core` design assumes the registry is effectively static after startup.

If you want live reload later, you will need extra design around:

- cache invalidation
- bean visibility
- consistency while requests are in flight
- revalidation of updated definitions

Recommended current approach:

- load once at startup
- restart or explicitly refresh the app when definitions change

That keeps behavior predictable.

Security Considerations

When using external sources, pay attention to:

- who can modify the YAML
- how changes are audited
- whether the YAML is versioned
- how secrets and credentials for the external system are managed

Because entity definitions influence:

- which persistence bean is used
- which modules run
- which audit, validator, and hook handlers are invoked

they should be treated as privileged framework configuration.

Recommended Architecture

For most applications, the best structure is:

- keep Coredeux YAML as the canonical entity-definition format
- keep `YamlEntityDefinitionLoader` unchanged
- create one application-specific provider that returns the YAML string
- create one application-specific `EntityDefinitionRegistry` bean

That gives you flexibility without changing `coredeux-core`.

Summary

To load `coredeux-entities.yml` from an external source:

1. fetch the complete YAML text as a `String`
2. convert it to `InputStream`
3. parse it with `EntityDefinitionLoader`
4. wrap the parsed definitions in `InMemoryEntityDefinitionRegistry`
5. expose that registry as the Spring bean used by Coredeux

This is the preferred current approach because it:

- reuses the existing YAML contract
- reuses the existing validation logic
- keeps `coredeux-core` unchanged
- keeps the external-source concern in the application layer

Previous: Entity Definitions | Documentation Home | Next: Module System